

Name: Monisha Raghini Chelle

RollNo:2520030537

Section-6

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

23.To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSLAB14      Simple-Commands-Week1  foldername  hello.c  monis  sample1.c
OSLAB                           OSSP-Practical  example.txt             hello       moni    sample1  sample1.c.save
monis@DESKTOP-C3GM4T6:~$ cd OSLAB
monis@DESKTOP-C3GM4T6:~/OSLAB$ ls
OSLAB28  Scheduling-Algorithms  hello  hello.c:Zone.Identifier  hello_link  newfile.txt  test1
Pipes    file1.txt              hello.c  hello:Zone.Identifier    hello_shortcut  skill        virtual_address
monis@DESKTOP-C3GM4T6:~/OSLAB$ cd skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 2 — Create the stress-testing script

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano stress_test.sh
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 stress_test.sh *
#!/usr/bin/env bash

set -u

REPORT="stress_report.txt"
FAILURES=0

{
    echo "Q23 - STRESS TEST REPORT"
    echo "Date: $(date)"
    echo "===== "
} > "$REPORT"

echo "TEST 1: Large pipeline" | tee -a "$REPORT"

if seq 1 100000 |
grep -E '[0-9]+$' |
awk '$1 % 2 == 0' |
sort -nr |
head -n 1000 |
wc -l > pipeline_output.txt
then
    RESULT=$(cat pipeline_output.txt)

    if [ "$RESULT" -eq 1000 ]; then
        echo "Pipeline: PASS" | tee -a "$REPORT"
    else
        echo "Pipeline: FAIL" | tee -a "$REPORT"
        FAILURES=$((FAILURES + 1))
    fi
else
    echo "Pipeline: FAIL" | tee -a "$REPORT"
    FAILURES=$((FAILURES + 1))
fi

echo "TEST 2: Multiple background jobs" | tee -a "$REPORT"

pids=()

for i in 1 2 3 4 5; do
    sleep 3 &
    pids+=("$!")
done

echo "Launched ${#pids[@]} jobs" | tee -a "$REPORT"

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 stress_test.sh *

echo "Launched ${#pids[@]} jobs" | tee -a "$REPORT"

for pid in "${pids[@]}; do
    if ! wait "$pid"; then
        echo "Job PID $pid: FAIL" | tee -a "$REPORT"
        FAILURES=$((FAILURES + 1))
    fi
done

echo "Background jobs finished." | tee -a "$REPORT"

echo "TEST 3: Performance and resources" | tee -a "$REPORT"

/usr/bin/time -v \
-o performance.txt \
bash -c 'seq 1 1000000 | wc -l' \
> performance_output.txt

if [ "$?" -eq 0 ]; then
    echo "Performance test: PASS" | tee -a "$REPORT"
else
    echo "Performance test: FAIL" | tee -a "$REPORT"
    FAILURES=$((FAILURES + 1))
fi

echo "TEST 4: Error detection" | tee -a "$REPORT"

if bash -c 'command_that_does_not_exist' \
> /dev/null 2> failure_output.txt
then
    echo "Error detection: FAIL" | tee -a "$REPORT"
    FAILURES=$((FAILURES + 1))
else
    echo "Error detection: PASS" | tee -a "$REPORT"
fi

echo "Total failures: $FAILURES" | tee -a "$REPORT"

if [ "$FAILURES" -eq 0 ]; then
    echo "FINAL RESULT: PASS" | tee -a "$REPORT"
else
    echo "FINAL RESULT: FAIL" | tee -a "$REPORT"
fi

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line  M-E Redo
```

Save using Ctrl + O, Enter, then Ctrl + X.

```
fi
echo "FINAL RESULT: FAIL" | tee -a "$REPORT"

Write to File: stress_test.sh
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse
^C Cancel    M-M Mac Format  M-P Prepend    ^O Discard buffer
```

Step 3 — Run the stress tests

chmod +x stress_test.sh

./stress_test.sh

cat stress_report.txt

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ chmod +x stress_test.sh
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./stress_test.sh
TEST 1: Large pipeline
Pipeline: PASS
TEST 2: Multiple background jobs
Launched 5 jobs
Background jobs finished.
TEST 3: Performance and resources
Performance test: PASS
TEST 4: Error detection
Error detection: PASS
Total failures: 0
FINAL RESULT: PASS
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ cat stress_report.txt
Q23 - STRESS TEST REPORT
Date: Sat Sep 26 23:38:20 IST 2026
=====
TEST 1: Large pipeline
Pipeline: PASS
TEST 2: Multiple background jobs
Launched 5 jobs
Background jobs finished.
TEST 3: Performance and resources
Performance test: PASS
TEST 4: Error detection
Error detection: PASS
Total failures: 0
FINAL RESULT: PASS
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ cat performance.txt
Command being timed: "bash -c seq 1 1000000 | wc -l"
User time (seconds): 0.01
System time (seconds): 0.07
Percent of CPU this job got: 126%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.06
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 8064
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 1240
Voluntary context switches: 486
Involuntary context switches: 1
Swaps: 0
File system inputs: 0
File system outputs: 8
Socket messages sent: 0
Socket messages received: 0
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ cat performance.txt
Command being timed: "bash -c seq 1 1000000 | wc -l"
User time (seconds): 0.01
System time (seconds): 0.07
Percent of CPU this job got: 126%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.06
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 8064
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 1240
Voluntary context switches: 486
Involuntary context switches: 1
Swaps: 0
File system inputs: 0
File system outputs: 8
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ cat failure_output.txt
bash: line 1: command_that_does_not_exist: command not found
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 4 — Document the shell architecture

Create your documentation file:

nano architecture.txt

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano architecture.txt
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 architecture.txt *
Q23 - SHELL ARCHITECTURE DOCUMENTATION

1. ARCHITECTURE

User Input
|
v
Command Parser
|
v
Syntax Validation
|
v
Built-in Command? ---- YES ---> Execute in Shell
|
NO
|
v
Create Child Process
|
v
Execute Command
|
+---- Foreground ---> Wait for Completion
|
+---- Background ---> Store Job Information
|
v
Monitor Job Status

2. EXECUTION PATHS

Foreground:
Read -> Parse -> fork -> exec -> waitpid -> Prompt

Background:
Read -> Parse -> fork -> exec -> Store PID -> Prompt

Pipeline:
Read -> Parse -> pipe -> fork children ->
dup2 -> exec -> close unused descriptors -> waitpid

3. IMPORTANT SYSTEM CALLS

fork()      Creates a child process.

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 architecture.txt *
+---- Background ----> Store Job Information
                        |
                        v
                        Monitor Job Status

2. EXECUTION PATHS

Foreground:
Read -> Parse -> fork -> exec -> waitpid -> Prompt

Background:
Read -> Parse -> fork -> exec -> Store PID -> Prompt

Pipeline:
Read -> Parse -> pipe -> fork children ->
dup2 -> exec -> close unused descriptors -> waitpid

3. IMPORTANT SYSTEM CALLS

fork()      Creates a child process.
execvp()    Executes a command.
pipe()      Creates a communication pipe.
dup2()      Redirects input or output.
waitpid()   Waits for or monitors child processes.
setpgid()   Assigns a process group.
tcsetgrp()  Transfers terminal control.
kill()      Sends signals to processes or groups.

4. DESIGN DECISIONS

- Separate parsing from command execution.
- Execute external commands in child processes.
- Use process groups for job control.
- Track background jobs using process IDs.
- Report errors without terminating the shell.
- Close unused file descriptors.
- Release resources after execution.

5. TESTING

Large pipelines: See stress_report.txt
Multiple jobs:  See stress_report.txt
Performance:   See performance.txt
Error detection: See failure_output.txt

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

Step 5 — Create the flow diagram

Use this flowchart in your Q23 record.

Shell execution flow

Read command
Parse and validate
Create process(es)

Foreground

Execute command
Wait for completion

Background

Store job information

Monitor status

Return to prompt

Step 6 — Final verification

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l stress_test.sh \
    stress_report.txt \
    performance.txt \
    failure_output.txt \
    architecture.txt
-rw-r--r-- 1 monis monis 1566 Sep 26 23:40 architecture.txt
-rw-r--r-- 1 monis monis   61 Sep 26 23:38 failure_output.txt
-rw-r--r-- 1 monis monis  746 Sep 26 23:38 performance.txt
-rw-r--r-- 1 monis monis  338 Sep 26 23:38 stress_report.txt
-rwxr-xr-x 1 monis monis 1879 Sep 26 23:37 stress_test.sh
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```